

Meta-learning with deep flow kernel network for few shot cross-domain remaining useful life prediction

Jing Yang, Xiaomin Wang*

School of Information Science and Technology, Southwest Jiaotong University, Chengdu, 610031, China

ARTICLE INFO

Keywords:

Prognostics and health management
Remaining useful life prediction
Few-shot
Cross-domain
Gaussian process
Contextual inference

ABSTRACT

Reliable prediction of the remaining useful life (RUL) is important for improving maintenance efficiency, equipment availability, and avoiding catastrophic accidents in complex industrial systems. Existing RUL prediction models have made some contribution, relying mainly on a large amount of degraded data with similar patterns or approximate distributions. However, in practical industrial systems, only a small amount of labeling data is usually available, which may also come from different devices and different working conditions, resulting in different distributions in the degraded data. This situation makes the existing RUL methods difficult to achieve satisfactory generalization performance. To address this challenge, this paper proposes a novel Meta-Learning with Deep Flow Kernel Network (MetaDFKN) model for RUL prediction under the few shot and cross-domain conditions. The model first learns kernel features in a data-driven manner and considers them as latent variables to improve the model's representative ability of shared knowledge between tasks. Then, we introduce the conditional normalized flow technique to infer richer posterior distributions in the kernel features, which helps to obtain feature information with stronger discriminative power. Moreover, shared knowledge and task-specific information are integrated into the contextual inference process, which can mine the dependencies of related tasks and capture richer domain information. Finally, to evaluate the proposed model, we conduct extensive experiments on engine and bearings degradation data, and the results verify the superiority of the model.

1. Introduction

Prognostics and health management (PHM) plays an essential role in modern industrial systems [1,2]. As the critical component of PHM, remaining useful life (RUL) prediction aims to estimate the exact time between the current health state of the machine and actual future failures [3]. To achieve accurate RUL prediction, numerous advanced solutions have been devised. Currently, mainstream methods are generally categorized into two categories: physics-modeled and data-driven [4]. Within the data-driven methods, deep learning (DL) methods serve as a critical branch [5,6]. It exhibits profound representation ability and can automatically learn critical features deeply hidden in the data, which helps to enhance RUL prediction accuracy and generalization performance [7,8].

At present, many DL methods have been employed for RUL prediction for various devices [9]. Among them, recurrent neural networks and their variant models are widely used. For example, the long short-term memory (LSTM) network is combined with a multi-stage RUL prediction model for aerospace engines [10]. The fuzzy c-means (FCM) and the bidirectional LSTM are integrated for RUL online prediction [11].

Besides, being a hot topic in DL methods, the convolutional neural network (CNN) is applied for RUL prediction of components [12]. For example, a deep convolutional multi-scale feature fusion RUL prediction model is proposed for aerospace engines [13]. Through a Bayesian CNN, the solenoid valve current features are trained to predict the remaining life of the solenoid valve (SV) [14]. Usually, existing DL methods are mainly developed from statistical learning, while a strong assumption of these methods is that degradation data should come from the same distribution [15].

However, during the degradation process of the devices, the degradation data collected from different devices or the same device under different operating conditions are very likely to show different data distributions with heterogeneity and imbalance due to the variation of setting values, operating conditions, mechanical wear, etc., which can be regarded as multi-domain data [16]. To address this issue, cross-domain analysis is proposed as a new learning strategy [17] to achieve RUL prediction by learning domain-invariant properties in a shared latent space. Specifically, by training a prediction model on data with specific working conditions (source domain) and then applying it to

* Corresponding author.

E-mail address: xmwang@swjtu.edu.cn (X. Wang).

data with different working conditions (target domain), the model's prediction performance in the target domain is improved by learning similar knowledge from the source domain [18–20].

At present, some methods for cross-domain prediction have been developed, these cross-domain RUL prediction methods are grouped as supervised learning and unsupervised learning [21,22]. In supervised learning-based cross-domain analysis, the models usually need to be trained by relying on source domain data and a limited number of labeled data from the target domain, and then the model is applied to the target domain for cross-domain prediction. Generally, the majority of RUL predictions primarily employ supervised cross-domain analysis methods [23]. For example, RUL cross-domain prediction on engine data is accomplished by using an adversarial domain adaptation method, which can maximize the mutual information between the input space and the latent space of the target domain data [24]. In [25], a weighted domain adversarial neural network based on Wasserstein distance for bearing RUL prediction is proposed, which uses domain classifiers to measure sample quality, and learns representations from samples with cross-domain similarity. These supervised learning-based prediction methods have made some progress in cross-domain analysis, but the limited amount of labeled data may not be sufficient to capture the complexity and variability of the target domain. It increases the difficulty of training the model, which leads to potential overfitting or underfitting problems.

While unsupervised learning-based cross-domain analysis focuses on training with completely unlabeled data in the target domain, it can better adapt to variations among the data and learn feature representations in the data, thus guiding the model to make cross-domain predictions in the target domain [26]. Currently, the application of unsupervised cross-domain analysis in RUL prediction is still a relatively new research topic [27]. For example, A combined model consisting of transformer and LSTM for engine RUL prediction is designed in [28], which narrows the domain gap by aligning distributions at the feature and semantic levels. In 2022, Miao *et al.* [29] designed a sparse domain adaptation network for bearing RUL prediction, it combines unsupervised sparse domain alignment and domain confrontation learning to address the problem of data distribution shift. These methods have made some improvements in unsupervised cross-domain prediction, but they require unlabeled samples from the target domain for model training, which still has some limitations in practical applications. This disparity arises from the inherent heterogeneity of the data, which can vary in format, structure, and granularity across different domains. Consequently, the application of unsupervised learning methods becomes intricate due to the unique preprocessing and feature extraction requirements imposed by individual datasets.

In addition, in existing cross-domain RUL prediction methods, large-scale labeled data are usually required for model training. However, in most practical applications, data collection and annotation are time-consuming and expensive [30]. In other words, only a tiny fraction of a large amount of data is labeled and available at any time, that is few-shot learning problem [31]. The most common solution is to use data augmentation techniques to expand the dataset by making various modifications to the data, such as adding Gaussian noise, masking noise, and signal translation [32,33]. To some extent, these methods can improve the model's recognition accuracy, but if the augmented data is too similar to the original data, which will result in overfitting. This could be a particular issue in cross-domain analysis, where augmented data might not accurately represent the target domain. Surprisingly, meta-learning [34] is a highly adaptable strategy that focuses on how to acquire the ability to learn, rather than the itself of learning task. It is good at leveraging the knowledge learned from previous tasks to improve performance on new and unseen tasks with few training examples [35,36].

Meta-learning has been widely studied in industrial system fault diagnosis, but it is still in the exploratory stage for remaining useful life prediction. Mo *et al.* [37] employs a model agnostic meta-learning

(MAML) algorithm to find suitable initialization of the model parameters, and then calculates the time series similarity to construct a pseudo-meta-rule task set of meta-learning for RUL prediction of the engine. Pan *et al.* [38] combines meta-learning and multi-head attentional networks for rule prediction, combining convolution-based and attentional branching for rocket engine RUL prediction. Further, an iterative meta-network pruning algorithm is proposed to automatically compute the meta-gradient of each convolutional kernel based on the chain rule to realize the RUL prediction for rocket motors [39]. These meta-learning-based models improve the RUL prediction accuracy to some extent, but may exhibit limited generalization ability when applied to datasets with significantly different features or under different operating conditions [40]. Besides, in extreme cases, data from the target domain may be unlabeled, while models trained only on the source domain may be biased towards the source domain, increasing prediction errors. Meanwhile, there may also be considerable variation in the data distribution between the source and target domains, making the model difficult to adapt to the new domain, resulting in poor prediction performance.

In order to address the aforementioned problems, this paper proposes a novel MetaDFKN model that is based on a **Meta**-learning with **Deep Flow Kernel Network** for few shot cross-domain RUL prediction. The key process is to combine the meta-Gaussian with the MAML framework to obtain the kernel feature through the data-driven method, on which it performs flow processing to capture more expressive feature information. Then, a tiny contextual inference network is designed by using fewer parameters to memorize shared knowledge and prior information from previous tasks, making it more sensitive to contextual information. Thereby, an effective RUL prediction in a few shot cross-domain scenario is realized.

The main contributions are summarized as follows:

- For the few shot cross-domain scenario, a MetaDFKN model based on the meta-learning framework is proposed for RUL prediction. In particular, we design a tiny contextual memory network to acquire rich domain information, which learns meta-knowledge through fast and accurate contextual inference on multiple related tasks. Because of this design, our model can use fewer parameters and quickly adapt to new tasks.
- The technique of conditional normalized flows is integrated into the Gaussian process regression model of cross-domain RUL prediction with fewer training samples, which can effectively enhance the expressiveness of variational posteriors. Thus, the model is guaranteed to capture critical information even in the few shot cross-domain scenario and produce accurate prediction results.
- Through the verification of extensive experiments the proposed MetaDFKN model shows superior performance than other advanced methods in cross-domain RUL prediction, demonstrating its great potential for solving complex few shot cross-domain RUL problems.

The remainder of this paper is structured as follows. Section 2 details the implementation of the MetaDFKN model. Section 3 introduces the two datasets and their preprocessing. Section 4 performs typical experiments and analysis to evaluate the MetaDFKN model and other advanced methods. The conclusions and future works are presented in Section 5.

2. Methodology

Taking the C-MAPSS dataset as an example, the overall architecture of MetaDFKN is illustrated in Fig. 1. On the left are different time series data collected from various sensors by monitoring the engine, and it is divided into the source domain and target domain. On the right, the Gaussian process regression model is used to model these

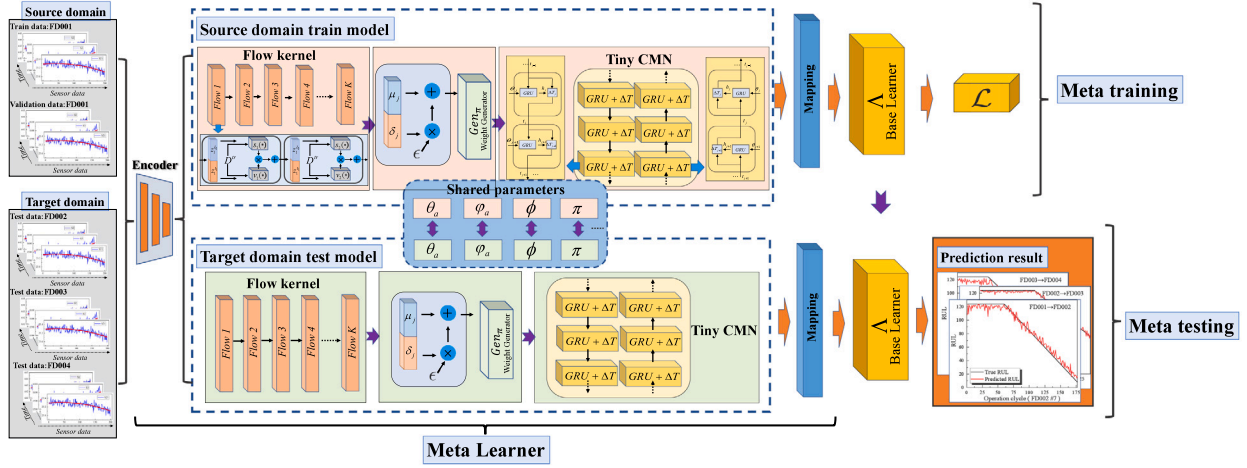


Fig. 1. The architecture of MetaDFKN.

tasks simultaneously. To build the kernel features contained in each sensor in the source domain, we add a set of latent variables with flow kernel under the meta-learning framework. To further infer the distributions of the latent variables with flow kernel, we employ a tiny contextual memory network (TinyCMN) that can extract the shared and task-specific knowledge. This way, the parallel processing of MetaDFKN in multiple RUL prediction tasks is realized, and the effective transfer of meta-knowledge is completed.

In Section 2.1, the concept of meta-learning and the setting of meta-Gaussian are explained to facilitate better understanding. Then, in Section 2.2, the meta-Gaussian is treated as a variational inference issue to introduce an expressive flow kernel. Finally, Section 2.3 gives the tiny contextual inference with the shared knowledge from relational tasks.

2.1. Meta-learning with Gaussian processes

As a meta-learning method based on optimization, the meta-Gaussian model is good at modeling nonlinear or complex properties of degradation data. For RUL prediction tasks, the meta-Gaussian is able to achieve transfer knowledge by learning the shared information and specific tasks information. In meta-learning, suppose there are totally T prediction tasks $\{\mathcal{T}_1, \dots, \mathcal{T}_T\}$ that are i.i.d. sampled from a task distribution $p(\mathcal{T})$. Each task comes with a dataset D_j that is randomly divided into a training dataset D_j^{tr} and a validation dataset D_j^{val} . We denote them by $D_j^{\text{tr}} = \{(x_i, y_i)\}_{i=1}^N$ and $D_j^{\text{val}} = \{(\tilde{x}_i, \tilde{y}_i)\}_{i=1}^M$. A Gaussian process regression (GPR) model [41] provides a method to perform nonlinear regression on a small amount of data, which can quickly achieve the intended purpose of the target model. Its representation function is

$$f(\cdot) = GP(0, k_\theta(\cdot)) \quad (1)$$

where θ denotes the kernel hyperparameters that control the Gaussian process regression model. For observed data $\{(x_n, y_n)\}_{n=1}^N$, additionally assuming the y data have an independent Gaussian noise:

$$y = f(x) + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma^2) \quad (2)$$

The covariance function $k_\theta(\cdot)$ is defined as a depth kernel function:

$$k(x, x' | \theta) = K(g_\theta(x), g_\theta(x')) \quad (3)$$

where g_θ is a feature extractor parameterized by θ . With the GPR defined, the marginal likelihood of $p(y)$ can be computed by integrating

out f and then performing a maximum likelihood estimation (MLE) based on a training set D_j^{tr} :

$$\max_{\theta} \int p(f_\theta) \prod_{(x,y) \in D_j^{\text{tr}}} p(y|f(x;\theta)) d f \quad (4)$$

The marginal likelihood is a closed form because all the distributions are assumed Gaussian. By optimizing the model parameter θ , we can find an MLE solution $\hat{\theta}$ that best fits the training data. The optimization procedure is usually a gradient descent algorithm.

2.2. Meta-learning with deep flow kernel network for RUL prediction

Since the degradation process of industrial equipment is usually affected by real factors like noise or operating conditions, and the degradation process of different equipment with the same source domain may show different data distributions, it is not appropriate to directly use meta-Gaussian for RUL prediction. We adopt the common training strategy under the model-agnostic meta-learning (MAML) framework, which includes meta-learner and base-learner. The meta-learner is responsible for synthesizing the experience of the base learner and giving the base learner the information it needs to achieve accurate predictions. In the training procedure of MetaDFKN, to obtain task-specific kernels we build a generative model with adaptive kernels in a data-driven manner. It can capture the shared knowledge between different prediction tasks and infer the latent features transferred among many tasks. To this end, assuming that $p(\tilde{y}|\tilde{x})$ has some underlying structure with a latent variable z :

$$\log \left[\prod_{i=1} p(\tilde{y}_i|\tilde{x}_i) \right] = \log \left[\prod_{i=1} p(\tilde{y}_i|z, \tilde{x}_i) p(z|\tilde{x}_i) d z \right] \quad (5)$$

where there are two additional assumptions: first, $p(\tilde{y}_i|\tilde{x}_i)$ is a multivariate Gaussian distribution; and second, $\tilde{y}_i$ and $\tilde{x}_i$ are independent conditioned on z . Thus, $p(\tilde{y}_i|z, \tilde{x}_i) = p(\tilde{y}_i|z)$ and is assumed a diagonal Gaussian $\mathcal{N}(0, \epsilon I)$.

Our goal is to learn the distribution $p(\tilde{y}_i|\tilde{x}_i)$. As direct learning is intractable, we adopt a variational approximation with a parametric model $p_\theta(\tilde{y}_i|\tilde{x}_i)$. That is, minimizing the KL-divergence $KL[p(\tilde{y}_i|\tilde{x}_i) \parallel p_\theta(\tilde{y}_i|\tilde{x}_i)]$ corresponds to maximizing $\log p_\theta(\tilde{y}_i|\tilde{x}_i)$. Adding a parameterized multivariate Gaussian latent prior $p_\theta(z|\tilde{x}_i)$, it can become

$$\log p_\theta(\tilde{y}_i|\tilde{x}_i) = \log \mathbb{E}_{p_\theta(z|\tilde{x}_i)} [p_\theta(\tilde{y}_i|z, \tilde{x}_i)] \quad (6)$$

Applying the variational evidence lower bounds (ELBO) and the learned variational multivariate Gaussian distribution $q_{\phi_a}(z|\tilde{x}_i, \tilde{y}_i)$, and with our independence assumption $p(\tilde{y}_i|z, \tilde{x}_i) = p(\tilde{y}_i|z)$, we can write

$f_\phi(\tilde{y}_i|z) = p(\tilde{y}_i|z, \tilde{x}_i)$. From this, Jensen's inequality is leveraged to obtain ELBO as

$$\mathcal{L} \geq \sum_{i=1}^T \left[\mathbb{E}_{q_{\phi_a}(z|D^{tr})} [\log f_\phi(\tilde{y}_i|z)] - \text{KL} [q_{\phi_a}(z|D^{tr}) \parallel p_{\theta_a}(z|\tilde{x}_i)] \right] \quad (7)$$

where the first term of Eq. (7) is the predicted log-likelihood based on the inferred GP latent variable. Maximizing it enables us to make precise predictions of RUL on the validation set. The meta-variational distribution $q_{\phi_a}(z|D^{tr})$ and the meta-prior distribution $p_{\theta_a}(z|\tilde{x}_i)$ in the second term are parameterized by θ_a, ϕ_a , respectively, which support the samples from training sets and validation sets to share the same GP.

However, for RUL prediction tasks, the posterior distribution $q_{\phi_a}(z|D^{tr})$ is a fully decomposed Gaussian distribution corresponding to a single kernel, which causes GP regression has constrained expressive capacity to approach the true posterior. Hence, the posterior is enriched by using the flow GP to meet RUL prediction requirements, which is also more expressive. Bayesian inference based on conditional normalized flow is used to infer the posterior distribution of network weights.

Through a series of transformations, flow GP can map the complex distribution $p_X(X)$ to the simple distribution $p_Z(Z)$. We make $x \in X$ sample $x \sim p_X(X)$ from sample data. Its core idea is to express $p_X(x)$ as the transformation $x = g(z)$ of a single gaussian distribution $z \sim p_Z = \mathcal{N}(0, I)$. In addition, assuming that the mapping relationship is bijective mapping $x = g(z) = f^{-1}(z)$. Hence, the log-likelihood of a complex distribution $p_X(x)$ can be defined as

$$\log p_X(x) = \log p_Z(f(x)) + \log \left(\left| \frac{\partial f(x)}{\partial x^T} \right| \right) \quad (8)$$

where $\left| \frac{\partial f(x)}{\partial x^T} \right|$ is the Jacobian determinant mapped by $f(x)$ at x .

In order to enable the Jacobian determinant mapped by $f(x)$ to process effectively and controllably, we divide the input $x \in \mathbb{R}^d$ into two components $x_1, x_2 \in \mathbb{R}^{d/2}$, which are inserted into the following transformation:

$$\begin{aligned} z_1 &= x_1 \\ z_2 &= x_2 \circ \exp(s_i(x_1)) + v_i(x_1) \end{aligned} \quad (9)$$

where $\circ$ is the element by element multiplication. $s_i(\cdot)$ and $v_i(\cdot)$ can be any complex mapping functions of x_i , and they need not be reversible. In practice, $s_i(\cdot)$ and $v_i(\cdot)$ are realized by neural network.

In the flow GP, there are K reversible flow blocks with staggered arrangements:

$$f = f_K \cdot r \dots f_2 \cdot r \cdot f_1 \quad (10)$$

where r is a displacement switch, which is used to replace the order of x_1 and x_2 . f is calculated by summing the Jacobian determinants of each f_i .

The conditional normalized flow is used to learn the conditional likelihood of complex target distribution in multivariate prediction tasks. Let $x \in X$ be the input and $y \in Y$ the regression target. Use conditional prior $p_{Z|X}(z|x)$ and mapping $f_\phi : Y \times X \rightarrow Z$ to learn complex distribution $p_{Y|X}(y|x)$, where Y and Z are bidirectional mappings. The logarithmic likelihood of this model is:

$$\log p_{Y|X}(y|x) = \log p_{Z|X}(f_\phi(y, x)|x) + \log \left(\left| \frac{\partial f_\phi(y, x)}{\partial y} \right| \right) \quad (11)$$

And different with Eq. (8), all distributions of Eq. (11) are conditional, and the conditional parameter is x .

In RUL prediction, let us assume that the length of flows is K , the flow parameterized approximate posterior distributions $q_{\phi_a}(z|D^{tr})$ is directly assigned by $q_K(z_K)$ after the K -step flow. The conditional affine coupling is applied to transform the distribution of latent variables. By extending the affine coupling layer, the Jacobian determinant mapped by $f(x)$ can be easily calculated.

$$z_K^{I_i} = z_{K-1}^{I_i} \odot \exp(s_{i+1}(z_{K-1}^{I_{1-i}}, D^{tr})) + v_{i+1}(z_{K-1}^{I_{1-i}}, D^{tr}) \quad (12)$$

where $i \in \{0, 1\}$, and transforming $s_{i+1}(\cdot)$ and $v_{i+1}(\cdot)$ do not need to be reversible, which are implemented by CNNs.

The logarithmic Jacobian determinant of a single affine coupling layer is calculated as the sum of $s_{i+1}(\cdot)$, that is $\sum_j s_1(z_{K-1}^{I_1}, D^{tr})_j + \sum_j s_2(z_K^{I_0}, D^{tr})_j$, j is the number of samples. This enables us to acquire more intricate distributions aligning with multi-kernel so that the model has competitive expressiveness.

Finally, the goal loss function by maximizing the logarithmic likelihood $\log p(\tilde{y}|\tilde{x}, D^{tr})$ of a batch of T tasks can be obtained:

$$\mathcal{L} = \frac{1}{T} \sum_{j=1}^T \left[\sum_{i=1}^I \left[\mathbb{E}_{q_0(z_0^j)} \left[\log f_\phi(\tilde{y}_i|z_K^j) - \sum_{k=1}^K \ln \left(\det \left| \frac{\partial f}{\partial z_k^j} \right| \right) \right] \right] + \mathbb{E}_{q_0(z_0^j)} \left[\log f_\phi(\tilde{y}_i|z_K^j) \right] \right] \quad (13)$$

For our few-shot regressions, we employ the latent encodes task-specific to instantiate the top-level weights of the regression task. This allows meta-learning in the latent space to adjust essential high-level parameters for regression tasks, instead of relying on the generator to provide high-dimensional parameters.

$$\mathcal{N}(\mu_j, \sigma_j^2) = g_{\phi_b}(z_K^j) \quad (14)$$

$$\eta_j \sim p(\eta_j|z_K^j) = \mathcal{N}(\mu_j, \sigma_j^2) \quad (15)$$

where $g_{\phi_b}(z_K^j)$ is the task contextual encoder. Eq. (15) is equivalent to generating η_j by the reparameterization trick

$$\eta_j = \mu_j + \sigma_j \odot \epsilon \quad (16)$$

where $\epsilon \sim \mathcal{N}(0, I)$, $\odot$ is element-wise multiplication [42].

The weight generator is trained to generate the functional weights for base-learner Λ_j conditioned on task contextual features. For each layer of the Λ_j , a conditional single-layer perceptron is constructed as the generator to produce weights:

$$\theta_j = \text{Gen}_\pi(\eta_j) \quad (17)$$

where $\text{Gen}_\pi(\cdot)$ is the weight generator. Therefore, the task experience of the base-learner is summarized through the meta-learner, and more accurate RUL prediction information is provided for the base learner. We refer to the RUL prediction model established by Deep Flow Kernel Network as DFKN.

2.3. Tiny contextual memory enhancement of DFKN

To perform flow kernel inference on current RUL prediction tasks, the tiny contextual memory network (TinyCMN) is designed, which can fully mine the contextual information of related tasks. For the TinyCMN, the GRU-based inference network is utilized to provide a concise and efficient structure for contextual inference by using its inherent ability to control information transmission.

In the current prediction task j , the input of TinyCMN is meta set parameter θ_j and tiny memory unit t_{j-1} , and the output is t_j . In order to avoid the influence of task order, we have formed an inference network based on bidirectional GRU, which can not only contain more contextual information, but also absorb knowledge about the previous prediction task and information about the next task [43]. The GRU hidden status updates are as follows

$$m_j = \text{sigmoid}(W_m \theta_j + U_m t_{j-1}) \quad (18)$$

$$r_j = \text{sigmoid}(W_r \theta_j + U_r t_{j-1}) \quad (19)$$

$$\tilde{h}_j = \tanh(W_h \theta_j + U_h (r_j \odot t_{j-1})) \quad (20)$$

$$h_j = m_j \odot t_{j-1} + (1 - m_j) \odot \tilde{h}_j \quad (21)$$

$$t_j = \Delta T_j(h_j, t_{j-1}) \quad (22)$$

where h_j is the updated hidden state, W_m, U_m, W_r, U_r, W_h are learnable parameters, $\text{sigmoid}(\cdot)$ and $\text{tanh}(\cdot)$ is activation function. m_j and r_j are the update and reset gates, respectively. ΔT_j is a tiny memory module, and the computation formula is:

$$t_j = \begin{cases} (h_j + t_{j-1})/2 & , \text{ if } \text{dist}(h_j, t_{j-1}) \geq \gamma \\ t_{j-1} & , \text{ if } \text{dist}(h_j, t_{j-1}) < \gamma \end{cases} \quad (23)$$

where h_j, t_{j-1} are the hidden states of current task and last task. $\text{dist}(\cdot)$ is the measurement function, and the Euclidean distance of h_j, t_{j-1} is calculated to judge the difference. When the difference between the hidden states at two times is greater than the threshold value γ , the experience of the first two tasks can be further considered; otherwise, only the experience of the current task can be considered. The tiny memory unit can selectively save previous time task experience, realize lightweight memory and effectively mitigate catastrophic forgetting of early data. The TinyCMN can be written as

$$\theta'_j = \text{Mer}(\theta_j, t_{j-1}; \varphi_c) \quad (24)$$

where the input of $\text{Mer}(\cdot)$ is meta set parameter θ_j and tiny memory unit t_{j-1} . φ_c is the learnable parameter. θ'_j is the weight value of base learner Λ_j obtained by inference.

Algorithm 1: MetaDFKN

Require: $\mathcal{T}_j$: Tasks drawn from distribution

Require: α, β : Inner-loop and outer-loop learning rates

```

1: Randomly initialize  $\Theta$ 
2: while not done do
3:   Sample task instance  $(D_j^r, D_j^{\text{val}}) \in \mathcal{T}_j$ 
4:   / ★ Outer-Loop Computing ★ /
5:   for iteration = 1, 2, ... do
6:     Encoder  $D_j^r$  to  $z$  using GP
7:     By  $K$  steps flow:  $z_K \leftarrow f_K \circ f_{K-1} \circ \dots \circ f_1(z_0)$ 
8:     Sample  $\eta_j \sim p(\eta_j | z_K) = N(\mu_j, \sigma_j^2)$ 
9:     Using  $\text{Gen}_\pi(\eta_j)$  initial params  $\theta_j$ 
10:     $\theta'_j \leftarrow \text{Mer}(\theta_j, t_{j-1}; \varphi_c)$ 
11:    / ★ Inner-Loop Computing ★ /
12:    for number of adaptation steps do
13:      Compute training loss  $\mathcal{L}^{(r)}(f_{\theta'_j})$ 
14:      Perform gradient:  $z'_j \leftarrow z'_j - \alpha \nabla_{z'} \mathcal{L}^{(r)}(f_{\theta'_j})$  obtain  $\theta'_j$ 
15:    end for
16:    Perform gradient step:  $\Theta \leftarrow \Theta - \beta \nabla_{\Theta} \mathcal{L}^{(\text{val})}(f_{\theta'_j})$ 
17:  end for
18: end while
```

In the learning process, the output of TinyCMN is transmitted across batches to make sure the information across all RUL prediction tasks can be accumulated in the meta-training set. Therefore, TinyCMN contains adequate prior knowledge extracted from all RUL prediction tasks experienced in the meta-training set. We call the core part MetaDFKN which is constructed by the flow kernel process inferred from the tiny contextual memory network. The entire algorithmic process is outlined in Algorithm 1.

3. Application on two datasets

This section introduces the details of the two datasets, data preprocessing, and evaluation metrics in the experiments.

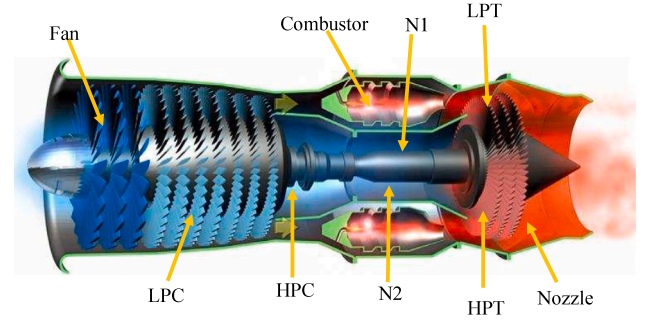


Fig. 2. The schematic diagram of aerospace engine.

Table 1
C-MAPSS dataset description.

Dataset	FD001	FD002	FD003	FD004
Training engines	100	260	100	249
Testing engines	100	259	100	248
Training samples	17 731	48 558	21 220	56 815
Testing samples	100	259	100	248
Max useful life	362	378	512	128
Min useful life	128	128	145	128
Average useful life	206	206	247	245

3.1. Overview of the datasets

3.1.1. C-MAPSS dataset

NASA widely uses the C-MAPSS dataset as a baseline for PHM [24]. As shown in Fig. 2, it is a multi-dimensional data set collected from multiple sensors to monitor the degradation process of an aircraft engine in real time. The data come from the low pressure compressor (LPC), the low pressure turbine (LPT), the high pressure compressor (HPC), the high pressure turbine (HPT), the fan and the combustion chamber in the engine. A total of 21 sensor devices are set up to monitor the speed, the temperature and the pressure, which are deployed at different collection points to track the degradation state of engines in real time.

The C-MAPSS dataset includes four datasets, Table 1 shows the detailed parameters of four datasets. FD001 and FD003 both have 100 training units and 100 test units, FD002 has 260 training units and 259 test units, and FD004 has 248 training units and 249 test units [24]. For every training unit, it is customary to produce a bit of initial wearing at the beginning. Then it starts to degrade gradually as the runtime increases, until it fails. For every test unit, the recording sequence will stop at some time before the system failure, so maintaining the actual RUL of the test unit. The training set composed of training units is used to train the prediction model, and the test set composed of test units is used for performance evaluation.

3.1.2. PRONOSTIA dataset

The PRONOSTIA dataset is sampled from the bearing test platform, which is applied for accelerated degradation experiments. As shown in Fig. 3, the platform contains load part, rotating part and data collection part. Among them, the PRONOSTIA test platform has a sample time of 0.1s, a sampling frequency of 25.6 kHz, a motor power of 250w, and 2560 data points are captured every 10 seconds [25]. The dataset is collected without adding human failures, and all of them are the natural degradation process of bearings, which can better match with the actual industrial scenarios. The entire accelerated degradation experiment includes almost all failure types such as inner race, roller, cage and outer race, and the detailed description of each experiment is shown in Table 2.

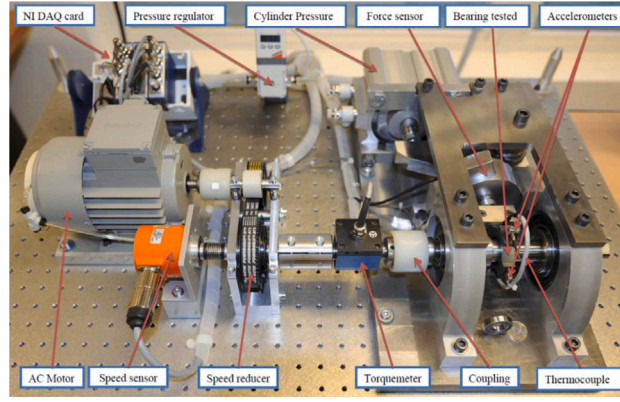


Fig. 3. PRONOSTIA test platform.

Table 2
PRONOSTIA dataset description.

Bearing1		Bearing2		Bearing3	
Load (N)	speed	Load (N)	speed	Load (N)	speed
4000	1800	4200	1650	5000	1500
Bearings	Actual lifetime	Bearings	Actual lifetime	Bearings	Actual lifetime
Bearing1_1	7 h 47 min 00 s	Bearing2_1	2 h 31 min 40 s	Bearing3_1	1 h 25 min 40 s
Bearing1_2	2 h 25 min 00 s	Bearing2_2	2 h 12 min 40 s	Bearing3_2	4 h 32 min 40 s
Bearing1_3	5 h 00 min 10 s	Bearing2_3	3 h 20 min 10 s	Bearing3_3	0 h 58 min 30 s
Bearing1_4	3 h 09 min 40 s	Bearing2_4	1 h 41 min 50 s		
Bearing1_5	6 h 23 min 29 s	Bearing2_5	5 h 33 min 30 s		
Bearing1_6	4 h 10 min 11 s	Bearing2_6	1 h 35 min 10 s		
Bearing1_7	6 h 23 min 30 s	Bearing2_7	0 h 28 min 30 s		

Table 3
The 12 cross-domain scenarios on C-MAPSS dataset.

Source	Target	Operating conditions	Fault types
FD001	FD002	One (Sea level) → Six	HPC degradation → HPC degradation
	FD003	One (Sea level) → One (Sea level)	HPC degradation → HPC and Fan degradation
	FD004	One (Sea level) → Six	HPC degradation → HPC and Fan degradation
FD002	FD001	Six → One (Sea level)	HPC degradation → HPC degradation
	FD002	Six → One (Sea level)	HPC degradation → HPC and Fan degradation
	FD003	Six → Six	HPC degradation → HPC and Fan degradation
FD003	FD001	One (Sea level) → One (Sea level)	HPC and Fan degradation → HPC degradation
	FD002	One (Sea level) → Six	HPC and Fan degradation → HPC degradation
	FD004	One (Sea level) → Six	HPC and Fan degradation → HPC and Fan degradation
FD004	FD001	Six → One (Sea level)	HPC and Fan degradation → HPC degradation
	FD002	Six → Six	HPC and Fan degradation → HPC degradation
	FD003	Six → One (Sea level)	HPC and Fan degradation → HPC and Fan degradation

3.2. Data preprocessing and metrics selection

3.2.1. C-MAPSS dataset preprocessing

In the 21 sensors, 14 sensors' data are highly correlated with the degradation process, showing a decreasing or increasing fluctuation trend [28]. Other sensor data fluctuate very weakly, which may hinder the learning of the degradation model. As a result, we only keep 14 sensor data sets with rich degradation information for model testing.

In the cross-domain research, the four datasets have various operating conditions and fault types [28]. Among them, FD001 has one operating condition (Sea level) and one fault type (HPC degradation), FD002 has six operating conditions and one fault type (HPC degradation), FD003 has one operating condition (Sea level) and two fault types (HPC and Fan degradation), FD004 has six operating conditions and two fault types (HPC and Fan degradation).

Table 3 presents the 12 cross-domain scenarios. FD001→FD002, FD002→FD001, FD003→FD004, and FD004→FD003 exhibit the transfer missions under various operating conditions. FD001→FD003, FD002→FD004, FD003→FD001, and FD004→FD002 exhibit the trans-

fer missions under various fault types. And other scenarios exhibit the transfer missions under various operating conditions and fault types. It is worth noticing that we utilize the training sets of FD001, FD002, FD003, and FD004 datasets as source domains to train our model, and make the test sets of the four datasets as target domains to test our model.

Considering that degradation may not have occurred at the beginning of the acquisition of the device, a segmented linear RUL in the form of [9] is used in the study. The degradation period of each bearing in the dataset is segmented according to a given value of RUL_{max} . When greater than the value of RUL_{max} are RUL_{max} , and conversely, the actual degradation value is maintained. The segmentation linearity is calculated as follows

$$RUL = \begin{cases} RUL & , RUL \leq RUL_{max} \\ RUL_{max} & , RUL > RUL_{max} \end{cases} \quad (25)$$

where RUL_{max} is a preset value of 125, keeping in line with most of Refs. [9].

Table 4

The 6 cross-domain scenarios on PRONOSTIA dataset.

Source	Target	Operating conditions	Loads(N)	Speeds(rpm)
Bearing1	Bearing2	One → Two	4000 → 4200	1800 → 1650
	Bearing3	One → Three	4000 → 5000	1800 → 1500
Bearing2	Bearing1	Two → One	4200 → 4000	1650 → 1800
	Bearing3	Two → Three	4200 → 5000	1650 → 1500
Bearing3	Bearing1	Three → One	5000 → 4000	1500 → 1800
	Bearing2	Three → Two	5000 → 4200	1500 → 1650

3.2.2. PRONOSTIA dataset preprocessing

The vibration signals of time series are used as samples in the experiments, and the vibration signals are the long-time sequence signal. It is intractable directly to learn the signal features for models, and the sliding time window is utilized to acquire signal samples. The window length and slide stride are set to 2560 as 2560 data points are collected every 10s [22].

In the cross-domain case, Bearing1, Bearing2 and Bearing3 have different loads, speeds, and operating conditions.

Table 4 shows the six cross-domain scenarios. Note that the training dataset contains only the data collected from the bearings in the source domain from operation to failure, while the test dataset contains only the data collected from the bearings in the target domain.

3.2.3. Evaluation metrics

Two metrics are used in this study to quantify MetaDFKN's performance for cross-domain RUL prediction. They are Root Mean Square Error (RMSE) and Score function (SCORE) [44]. The RMSE metric is defined as follows:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2} \quad (26)$$

where $\hat{y}_i$ and y_i are the predicted values by models and true values, respectively. N is the number of samples.

RMSE evaluates all RUL predictions equally, and the SCORE function that is proposed in the PHM 2008 Data Challenge [45] is able to impose severe penalties on RUL predictions with large predicted values. When the predicted value is greater than the true value, this means that the model underestimated the severity of the failure and predicted that the equipment would last longer than actual results. When the predicted value is less than the true value, this means that the model overestimates the severity of the failure and predicts that the equipment will fail sooner than actual results. In practice, the penalty for predicted value less than the true value is relatively small compared to predicted value more than the true value, because it is preferable to prepare for equipment failure in advance rather than be caught off guard by a sudden breakdown.

$$S_{\text{engine}} = \begin{cases} \frac{1}{N} \sum_{i=1}^N (e^{-\frac{\hat{y}_i - y_i}{13}} - 1), & \text{if } (\hat{y}_i < y_i) \\ \frac{1}{N} \sum_{i=1}^N (e^{\frac{\hat{y}_i - y_i}{10}} - 1), & \text{if } (\hat{y}_i \geq y_i) \end{cases} \quad (27)$$

The S_{engine} of C-MAPSS dataset is different from the S_{bearing} of PRONOSTIA dataset, the former is that lower values imply better properties, while the latter has opposite effect [46].

$$S_{\text{bearing}} = \begin{cases} \frac{1}{N} \sum_{i=1}^N (e^{\frac{-\ln(0.5) \times (y_i - \hat{y}_i) \times 100}{5 \times y_i}}), & \text{if } (y_i \leq \hat{y}_i) \\ \frac{1}{N} \sum_{i=1}^N (e^{\frac{\ln(0.5) \times (y_i - \hat{y}_i) \times 100}{20 \times y_i}}), & \text{if } (y_i > \hat{y}_i) \end{cases} \quad (28)$$

4. Experiments and result analysis

This section evaluates the RUL prediction performance of our MetaDFKN model in terms of RMSE and SCORE metrics by comparing it with the existing models.

4.1. Experimental parameters analysis

4.1.1. Flow kernel lengths

For the MetaDFKN model, the flow lengths K are set as 2, 4, 6, 8, 10, 12, 14, 16, 18, and 20 to analyze the prediction accuracy. Since the experimental results of the PRONOSTIA dataset are roughly similar to those of the C-MAPSS dataset, i.e., the K values are selected consistently. Hence, the experimental results of 12 cross-domain scenarios in the C-MAPSS dataset are selected for presentation, as shown in Fig. 4.

It can be observed from Fig. 4 there are fluctuations in RMSE and SCORE as K changes, rather than a strict monotonic increase followed by a monotonic decrease. This is caused by the random initialization parameters of the meta-learning model and differences in data distribution. Besides, it is found that the MetaDFKN model shows excellent performance at $K = 10$ in most scenarios. As the value of K increases, the computational resources and time of the model gradually increase due to the increase in the layers of flow kernel and the model parameters, but the prediction effect of the model does not improve significantly. Therefore, we finally choose $K = 10$ as the optimal parameter for flow kernel length.

4.1.2. Number of network layers

The hidden layers of TinyCMN L are set as 1, 3, 5, 7, and 9 to analyze our model performance. Similarly, since the experimental results of the PRONOSTIA dataset are roughly similar to those of the C-MAPSS dataset, the experimental results of the 12 cross-domain scenarios in the C-MAPSS dataset are selected for presentation. Fig. 5 shows the experimental results of the RMSE and SCORE with the increase of the hidden layer (from 1 to 9). It can be observed that the property of TinyCMN is in the decrease state as the increase of hidden layer L , but there are some differences under different scenarios. For the scenarios of FD001→FD004 and FD004→FD001, the curves change sharply because the FD001 dataset has a single fault type and the FD004 dataset has two fault types. Additionally, the two datasets have significant differences, and the limited training data is easy to cause overfitting, leading to the prediction performance decreases. Hence, we finally use the single GRU layer considering the property of our model.

4.2. Ablation experiments analysis

In order to evaluate the contribution of every component in proposed MetaDFKN model for RUL prediction, we carry out the ablation experiments. Specifically, two variants of MetaDFKN are derived, one is $MN1$, which is MetaDFKN without the tiny contextual memory network (TinyCMN), and another is $MN2$, which is MetaDFKN without flow kernel inference. Figs. 6 and 7 show the cross-domain experimental results on the C-MAPSS and PRONOSTIA datasets.

In Figs. 6 and 7, it can be found that almost all the RUL percentages estimated by the MetaDFKN model are within 20% error margin under different cross-domain scenarios. And as the engine nears failure, the prediction becomes more accurate. In contrast, the prediction accuracy of $MN1$ and $MN2$ methods is lower in most cases, and the prediction effect is not stable with different time steps. Obviously, the performance of $MN2$ model is the worst, implying the data distribution of source and target domains is quite different, and the absence of flow kernel results in the inability to capture the more complex distributions corresponding to multi-kernel, so that the model's prediction effect is not satisfied. Besides, the prediction effect of the proposed MetaDFKN model is more outstanding than that of $MN1$, demonstrating the TinyCMN can learn the experience of multiple tasks well, and

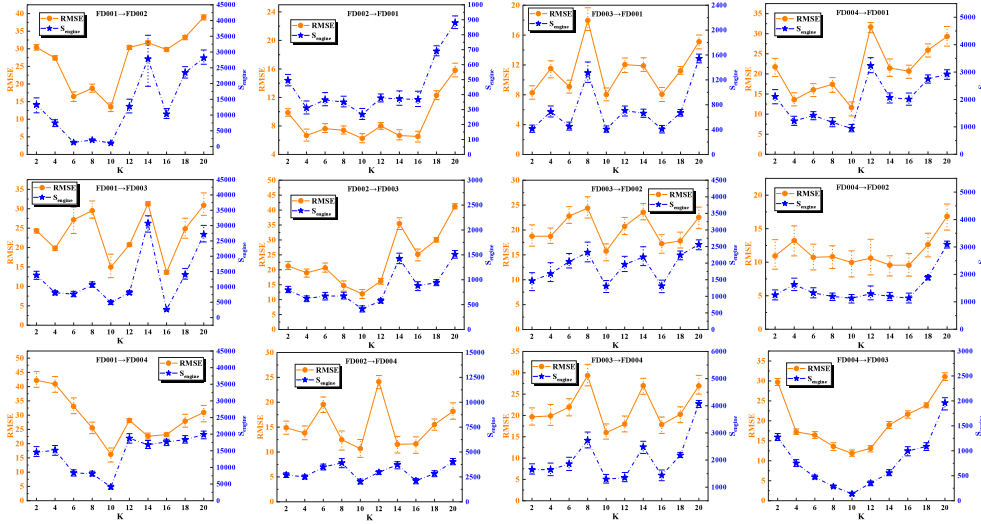


Fig. 4. Experimental results of MetaDFKN with different K values in 12 cross-domain scenarios in the C-MAPSS dataset.

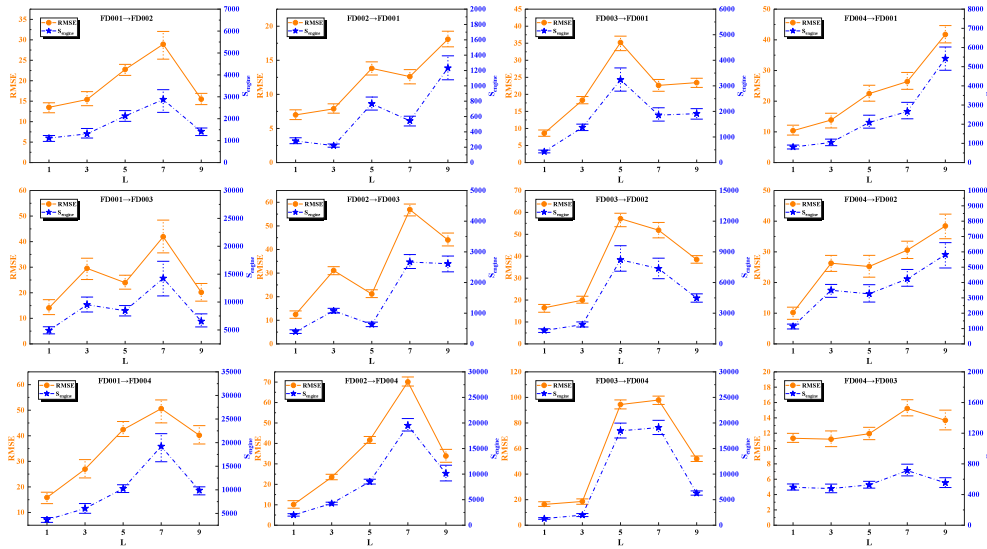


Fig. 5. Experimental results of MetaDFKN with different L values in 12 cross-domain scenarios in the C-MAPSS dataset.

Table 5

Detailed introduction for 11 RUL prediction methods.

No.	Method's name	Method's type	Implementation
1	Deutsch's Method [4]	No	DBN
2	Mao's Method [5]	Transfer	LSTM
3	Zhu's Method [6]	Learning	Multi-Scale CNN
4	Kinney's Method [18]	Shallow	Kernel Mean Matching
5	Fernando's Method [19]	Transfer	Subspace Alignment
6	S. J. Pan's Method [20]	Learning	Transfer Component Analysis
7	Ganin's Method [21]		DANN
8	Costa's Method [22]	Deep	LSTM-DANN
9	Finn's Method [34]	Transfer	MAML
10	Mo's Method [37]	Learning	Improved MAML
11	Pan's Method [38]		Improved MAML

comprehensively learn global contextual information. Therefore, the MetaDFKN model is capable of efficiently learning invariant features from various cross-domain scenarios, and effectively characterizing the equipment's health degradation trend.

4.3. Sensitivity analysis

The dataset used in this study is not pure complete life-cycle degradation data, and they contain some degree of noise. To better Simulate the real uncertain noise and to explore the effect of noise on RUL prediction, Gaussian white Noise (WGN) with different Signal-to-Noise Ratio (SNR) levels is added to the complete life-cycle degradation data of the C-MAPSS Dataset. The SNR is defined as

$$SNR_{dB} = 10 \log_{10} \left(\frac{P_{signal}}{P_{noise}} \right) \quad (29)$$

where P_{signal} and P_{noise} are the original degraded source data and the added noise signal, respectively. Three SNR scenarios (−6 dB, 0 dB and 6 dB) are set, and the engine data are used to conduct experiments under intense, medium, and weak noise conditions, respectively. Furthermore, considering the possible differences in RUL predictions for different N -shot, i.e., randomly taking N dimensions data from 14 dimensions as input, we set N values of 3, 5, 10 and 14, respectively.

As shown in Fig. 8, MetaDFKN achieves better prediction performance in different cross-domain scenarios for each SNR and N -shot scenarios. Evidently, the variation of RMSE and SCORE of MetaDFKN model under the influence of different noises in the same scenario is

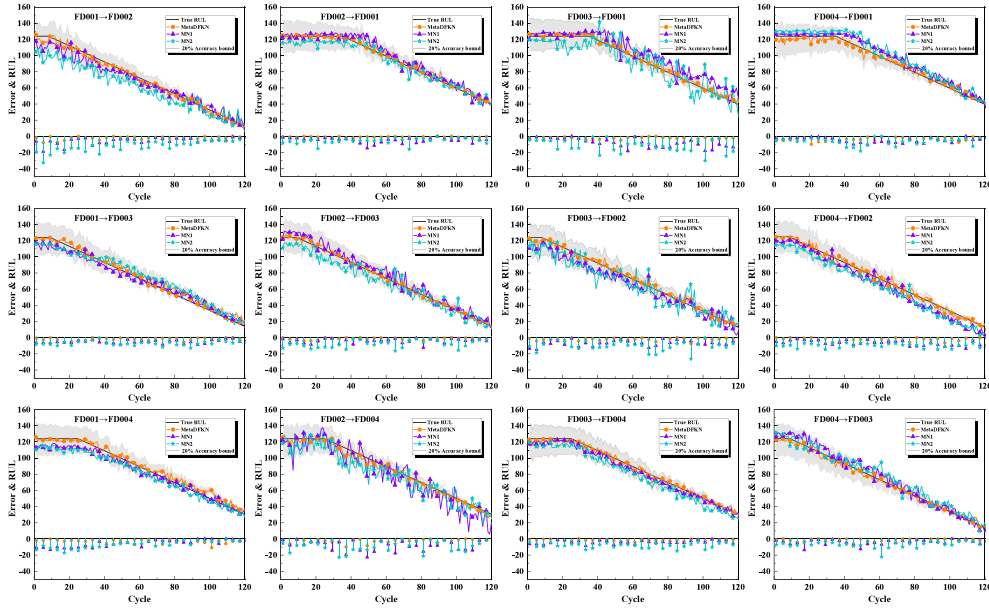


Fig. 6. Ablation experiments in 12 cross-domain cross-domain scenarios of C-MAPSS dataset.

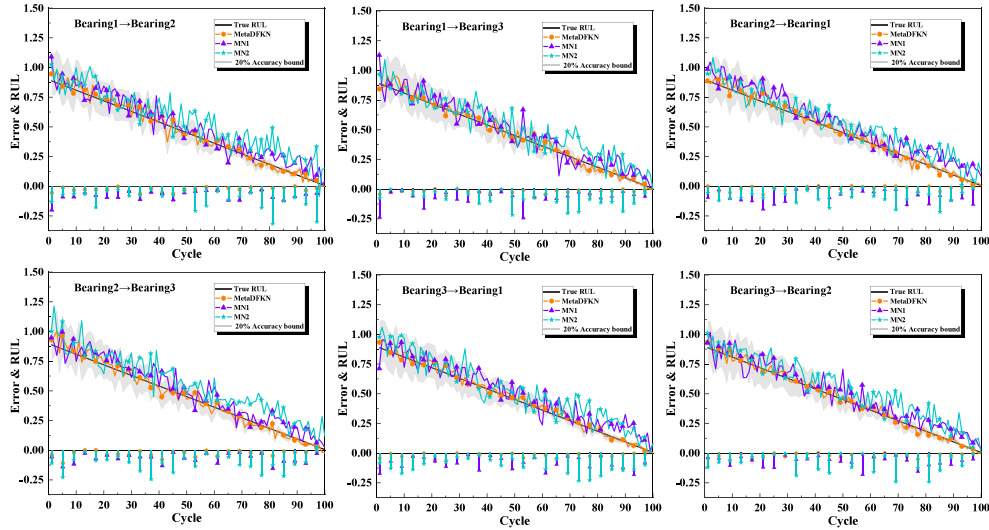


Fig. 7. Ablation experiments in 6 cross-domain cross-domain scenarios of PRONOSTIA dataset.

relatively minor, indicating that the model has excellent noise resistance. This is because the flow kernel can extract richer and more comprehensive specific knowledge and adaptively perform fusion to optimize the specific knowledge of different tasks, which provides more effective learning parameters for the base learner and enables the base learner to obtain more accurate prediction results. In addition, the RMSE and SCORE obtained in the same scenario show a decreasing trend as the N value increases. This is because the model acquires more information during the training process. The variation of the same N value in different scenarios is more prominent, which is due to the significant difference in the information contained between different datasets, resulting in a large variation of the model prediction performance at the same N value.

4.4. RUL prediction in source-only data driven

In industrial system analysis, there are some cases of few-shot analysis using cross-domain learning with partial data of the target domain. However, the model is trained exclusively on data from the

source domain and evaluated on the target domain throughout the knowledge transfer process, creating an intriguing analytical scenario known as “Source-Only”. To better present the prediction effect of our model, the proposed MetaDFKN is compared with eleven representative and popular RUL prediction methods, as shown in Table 5.

For brevity, Figs. 9 and 10 show the RUL prediction results for few shot cross-domain on the C1 scenario of C-MAPSS dataset and the D1 scenario of PRONOSTIA dataset, respectively. The prediction and visual effects of MetaDFKN in cross-domain scenarios for both datasets are significantly better than the existing RUL prediction methods. Specifically, the methods (No. 1–3) without transfer learning cannot learn domain knowledge effectively. Shallow transfer learning methods (No. 4–6), although employing a domain feature transfer strategy, cannot be well suited for RUL predictions trained with source domain data only. The deep transfer learning methods (No. 7–11) are still inferior to our model.

For deep migration learning methods, meta-learning methods based on the MAML framework (No. 9–11) have significant advantages over

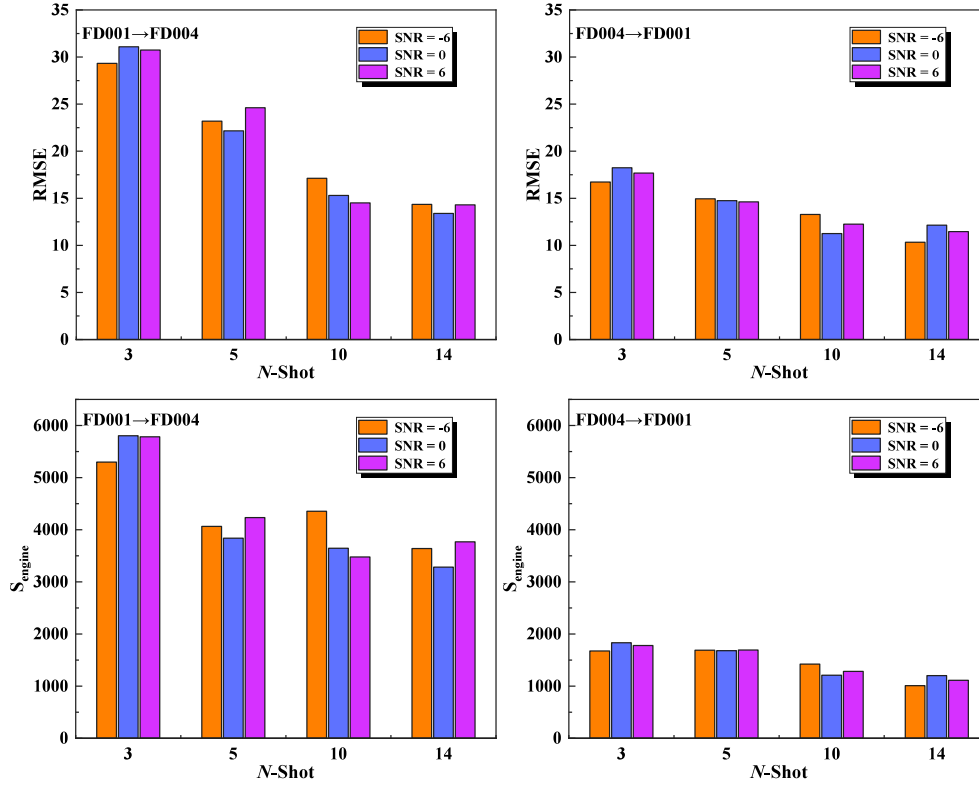


Fig. 8. Different SNR sensitivity experiments in partial cross-domain scenarios of C-MAPSS dataset.

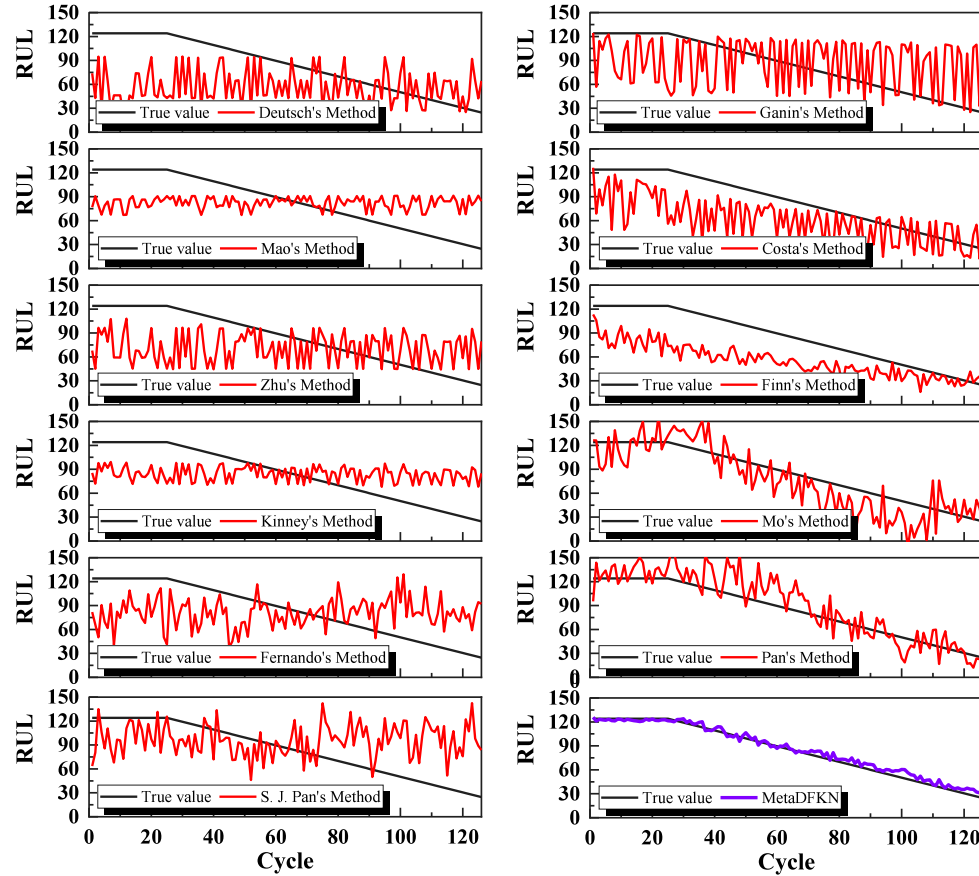


Fig. 9. RUL prediction results in Source-Only data driven on C-MAPSS dataset.

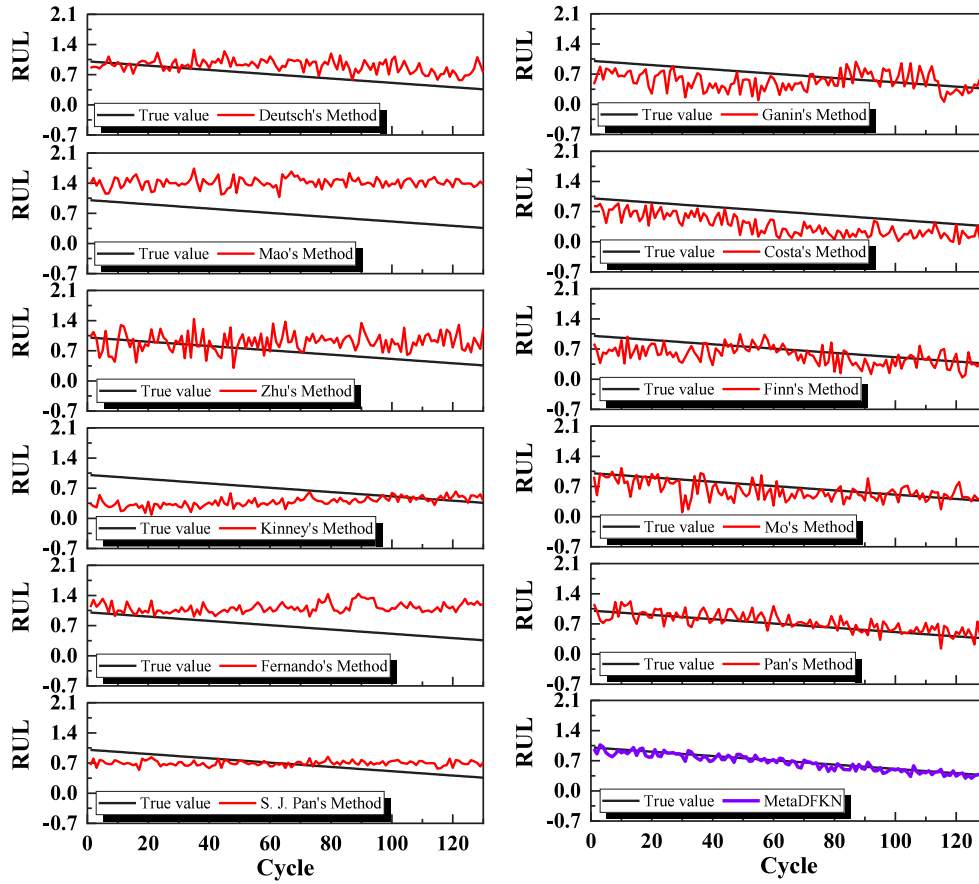


Fig. 10. RUL prediction results in Source-Only data driven on PRONOSTIA dataset.

deep neural network methods (No. 7–8). This is because in the knowledge transfer process, the meta-learning model is directly capable of updating the initialization parameters obtained from the pretraining stage. Transfer learning is originally intended to minimize the loss sum across all tasks, while deep neural networks are concerned only with the current task's performance and cannot ensure optimal training across all tasks. The meta-learning models adapted to RUL prediction (No. 10–11) showed better prediction results than the MAML method (No. 9) in the few shot cross-domain analysis under the meta-learning framework. While, the proposed MetaDFKN model significantly outperforms the meta-learning models adapted to RUL prediction (No. 10–11). This is because the MetaDFKN model treats the learned kernel features as latent variables in a data-driven manner, and then introduces a conditional normalization flow in the kernel features to infer richer posterior distributions, thus obtaining feature information with stronger discriminative power. Therefore, our model has a better few shot cross-domain RUL prediction effect.

4.5. Comparison with recently reported methods

In this part, we compare the proposed MetaDFKN model with existing advanced methods to confirm that our model is effective, including RULDDA [22], CORAL [23], CADA [24], WD-WDANN [25], DATransformer [28], SDAN [29], DIDRLSTM [45], MTLTrans [47] and TMSATCN [48]. According to the experimental setup scheme of the above latest methods, these models are first trained with the dataset of the source domain, and then they are fine-tuned with a limited quantity of data from the target domain, as “Target-Unsupervised”. The experimental parameters and results of existing methods come from Refs. [22–25,28,29,45,47,48].

Using RMSE and SCORE as evaluation metrics, the performance of different methods is evaluated in different cross-domain scenarios.

Tables 6 and 7 present the test results on the C-MAPSS dataset.

Tables 8 and 9 present the test results on the PRONOSTIA dataset. In general, the proposed MetaDFKN model is more outstanding than other advanced methods. Furthermore, we analyzed the best method among the reported methods and compared its results with MetaDFKN. On the C-MAPSS dataset, the average improvement rate results demonstrate that the MetaDFKN model outperforms the DATransformer method by over 36% and 55% on RMSE and SCORE, respectively. On the PRONOSTIA dataset, we compare the results of TMSATCN method and the RMSE and SCORE of MetaDFKN model exceed 86% and 58%, respectively. Through the above comparisons, the MetaDFKN model can transfer knowledge between complex data sets of different degrees and show excellent performance in cross-domain RUL prediction.

5. Conclusions

This paper presents a novel MetaDFKN model based on meta-learning to address the few shot cross-domain problem in RUL prediction. The model can effectively realize few shot cross-domain prediction, and has strong knowledge transfer adaptability. The model gains the shared knowledge in the latent space in the form of implicit modeling, and obtains a more expressive flow kernel through flow processing. Second, we design a tiny contextual inference method that integrates prior shared knowledge and task-specific information into the current task. Then, the previous shared knowledge and prior information are memorized by the tiny inference network. Finally, with the help of probabilistic modeling, a new RUL prediction about the few shot cross-domain task is learned by using the variational posterior and meta-knowledge. The effectiveness of the model is validated with the publicly available C-MAPSS and PRONOSTIA datasets. Experimental results show that the proposed MetaDFKN model has superior performance in few shot cross-domain RUL prediction. In the future, we will

Table 6

RMSE results of MetaDFKN and the recently reported methods on the C-MAPSS dataset.

Source	Target	RULDDA	DIDRLSTM	MTLTrans	CADA	DATransformer	MetaDFKN	Imp
FD001	FD002	24.08	29.40	16.63	19.52	15.21	9.26	39.12%
	FD003	43.08	20.90	20.83	39.58	15.50	13.03	15.94%
	FD004	45.70	34.40	21.76	31.23	15.89	13.11	17.49%
FD002	FD001	23.91	16.10	13.67	13.88	11.29	3.65	67.67%
	FD003	47.26	21.10	20.60	33.53	13.76	10.91	20.71%
	FD004	45.17	34.00	22.18	33.71	12.16	9.38	22.86%
FD003	FD001	27.15	18.80	21.22	19.54	10.56	5.48	48.11%
	FD002	30.42	29.40	23.79	19.33	17.08	14.93	12.58%
	FD004	31.82	32.30	21.04	20.61	17.05	9.45	44.58%
FD004	FD001	32.37	21.30	18.44	20.10	11.73	5.12	56.35%
	FD002	27.54	28.90	16.86	18.50	12.01	7.81	34.97%
	FD003	23.31	16.90	13.56	14.49	14.83	6.15	58.53%

Note: Imp represents the improvement rate of the proposed MetaDFKN with the DATransformer method.

Table 7

SCORE results of MetaDFKN and the recently reported methods on the C-MAPSS dataset.

Source	Target	RULDDA	DIDRLSTM	MTLTrans	CADA	DATransformer	MetaDFKN	Imp
FD001	FD002	2684	4822	1618	2122	1113	878	21.11%
	FD003	10 259	470	2286	8415	2794	1454	47.96%
	FD004	26 981	5538	4714	11 577	1339	908	32.19%
FD002	FD001	2430	213	331	351	539	140	74.03%
	FD003	12 756	474	1510	5213	1919	372	80.62%
	FD004	25 738	5471	3860	15 106	1031	701	32.01%
FD003	FD001	2931	442	1890	1451	664	226	65.96%
	FD002	6754	4127	9098	5257	1855	922	50.30%
	FD004	5775	4123	13 261	3219	1879	1030	45.18%
FD004	FD001	1337	553	1096	1840	629	213	66.14%
	FD002	4937	3483	2189	4460	847	383	54.78%
	FD003	1679	249	383	682	1972	112	94.32%

Table 8

RMSE results of MetaDFKN and the recently reported methods on the PRONOSTIA dataset.

Source	Target	CORAL	SDAN	RULDDA	WD-WDANN	TMSATCN	MetaDFKN	Imp
Bearing1	Bearing2	0.724	0.398	0.160	0.089	0.120	0.007	94.17%
	Bearing3	0.331	0.064	0.190	0.138	0.090	0.018	80.00%
Bearing2	Bearing1	0.712	0.353	0.280	0.184	0.130	0.012	90.77%
	Bearing3	0.342	0.098	0.240	0.151	0.120	0.011	90.83%
Bearing3	Bearing1	0.557	0.291	0.750	0.136	0.150	0.024	84.00%
	Bearing2	0.728	0.421	0.730	0.114	0.120	0.026	78.33%

Note: Imp represents the improvement rate of the proposed MetaDFKN with the TMSATCN method.

Table 9

SCORE results of MetaDFKN and the recently reported methods on the PRONOSTIA dataset.

Source	Target	CORAL	SDAN	RULDDA	WD-WDANN	TMSATCN	MetaDFKN	Imp
Bearing1	Bearing2	0.301	0.541	0.510	0.525	0.520	0.707	35.96%
	Bearing3	0.339	0.630	0.490	0.408	0.530	0.743	40.19%
Bearing2	Bearing1	0.298	0.556	0.430	0.394	0.490	0.809	65.10%
	Bearing3	0.310	0.621	0.440	0.436	0.500	0.810	62.00%
Bearing3	Bearing1	0.287	0.587	0.250	0.358	0.370	0.716	93.51%
	Bearing2	0.195	0.562	0.260	0.389	0.460	0.706	53.48%

focus on more extensive and far-reaching research on the unsupervised RUL prediction of cross-domain. It is also interesting to further study domain knowledge fusion based on meta-learning and RUL prediction.

CRedit authorship contribution statement

Jing Yang: Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Xiaomin Wang:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work is supported by the National Natural Science Foundation of China under Grant 61872302, in part by the Key R&D program for Railway Science and Technology of China under Grants P2021G053 and L2022G004, in part by the Sichuan Provincial Natural Science Foundation under Grant 2022NSFSC1878.

References

- [1] Mitici M, de Pater I, Barros A, Zeng Z. Dynamic predictive maintenance for multiple components using data-driven probabilistic RUL prognostics: The case of turbofan engines. *Reliab Eng Syst Saf* 2023;234:109199.

- [2] Zuo T, Zhang K, Zheng Q, Li X, Li Z, Ding G, et al. A hybrid attention-based multi-wavelet coefficient fusion method in RUL prognosis of rolling bearings. *Reliab Eng Syst Saf* 2023;237:109337.
- [3] Zhang H, Xi X, Pan R. A two-stage data-driven approach to remaining useful life prediction via long short-term memory networks. *Reliab Eng Syst Saf* 2023;237:109332.
- [4] Deutsch J, He D. Using deep learning-based approach to predict remaining useful life of rotating components. *IEEE Trans Syst, Man, Cybern: Syst* 2018;48:11–20.
- [5] Mao W, He J, Tang J, Li Y. Predicting remaining useful life of rolling bearings based on deep feature representation and long short-term memory neural network. *Adv Mech Eng* 2018;10.
- [6] Zhu J, Chen N, Peng W. Estimation of bearing remaining useful life based on multiscale convolutional neural network. *IEEE Trans Ind Electron* 2019;66:3208–16.
- [7] Zhang J, Li X, Tian J, Luo H, Yin S. An integrated multi-head dual sparse self-attention network for remaining useful life prediction. *Reliab Eng Syst Saf* 2023;233:109096.
- [8] Xu Q, Chen Z, Wu K, Wang C, Wu M, Li X. KDnet-RUL: A knowledge distillation framework to compress deep neural networks for machine remaining useful life prediction. *IEEE Trans Ind Electron* 2022;69:2022–32.
- [9] Zhang X, Sun J, Wang J, Jin Y, Wang L, Liu Z. PAOLTransformer: Pruning-adaptive optimal lightweight transformer model for aero-engine remaining useful life prediction. *Reliab Eng Syst Saf* 2023.
- [10] Liu J, Lei F, Pan C, Hu D, Zuo H. Prediction of remaining useful life of multi-stage aero-engine based on clustering and LSTM fusion. *Reliab Eng Syst Saf* 2021;214:107807.
- [11] Chen C, Lu N, Jiang B, Xing Y, Zhu ZH. Prediction interval estimation of aeroengine remaining useful life based on bidirectional long short-term memory network. *IEEE Trans Instrum Meas* 2021;70:1–13.
- [12] Ding W, Li J, Mao W, Meng Z, Shen Z. Rolling bearing remaining useful life prediction based on dilated causal convolutional DenseNet and an exponential model. *Reliab Eng Syst Saf* 2022;232:109072.
- [13] Li X, Jiang H, Liu Y, Wang T, Li Z. An integrated deep multiscale feature fusion network for aeroengine remaining useful life prediction with multisensor data. *Knowl-Based Syst* 2022;235:107652.
- [14] Mazaev T, Crevecoeur G, Hoecke SV. Bayesian convolutional neural networks for remaining useful life prognostics of solenoid valves with uncertainty estimations. *IEEE Trans Ind Inf* 2021;17(12):8418–28.
- [15] Cheng H, Kong X, bin Wang Q, Ma H, Yang S. The two-stage RUL prediction across operation conditions using deep transfer learning and insufficient degradation data. *Reliab Eng Syst Saf* 2022;225:108581.
- [16] Xing S, Lei Y, Yang B, Lu N. Adaptive knowledge transfer by continual weighted updating of filter kernels for few-shot fault diagnosis of machines. *IEEE Trans Ind Electron* 2022;69(2):1968–76.
- [17] Xia P, Huang Y, Li P, Liu C, Shi L. Fault knowledge transfer assisted ensemble method for remaining useful life prediction. *IEEE Trans Ind Inf* 2022;18(3):1758–69.
- [18] Kinney J, Atwal G. Correcting sample selection bias by unlabeled data. In: *International conference on neural information processing systems*. 2006.
- [19] Fernando B, Habrard A, Sebban M, Tuytelaars T. Unsupervised visual domain adaptation using subspace alignment. In: *International conference on computer vision*. 2013, p. 2960–7.
- [20] Pan SJ, Tsang IW-H, Kwok JT-Y, Yang Q. Domain adaptation via transfer component analysis. *IEEE Trans Neural Netw* 2009;22:199–210.
- [21] GaninYaroslav, UstinovaEvgeniya, AjakanHana, GermainPascal, LarochelleHugo, LavioletteFrançois, et al. Domain-adversarial training of neural networks. *J Mach Learn Res* 2016.
- [22] da Costa P, Akçay AE, Zhang Y, Kaymak U. Remaining useful lifetime prediction via deep domain adaptation. *Reliab Eng Syst Saf* 2020;195:106682.
- [23] Sun B, Feng J, Saenko K. Correlation alignment for unsupervised domain adaptation. 2016, ArXiv, arXiv:1612.01939.
- [24] Ragab M, Chen Z, Wu M, Foo C-S, Keong KC, Yan R, et al. Contrastive adversarial domain adaptation for machine remaining useful life prediction. *IEEE Trans Ind Inf* 2021;17:5239–49.
- [25] Hu T, Guo Y, Gu L, Zhou Y, Zhang Z, Zhou Z. Remaining useful life estimation of bearings under different working conditions via wasserstein distance-based weighted domain adaptation. *Reliab Eng Syst Saf* 2022;224:108526.
- [26] Mao W, Chen J, Liu J, Liang X. Self-supervised deep domain-adversarial regression adaptation for online remaining useful life prediction of rolling bearing under unknown working condition. *IEEE Trans Ind Inf* 2023;19:1227–37.
- [27] Jiang Y, Xia T, Wang D, Fang X, Xi L. Adversarial regressive domain adaptation approach for infrared thermography-based unsupervised remaining useful life prediction. *IEEE Trans Ind Inf* 2022;18:7219–29.
- [28] Li X, Li J, Zuo L, Zhu L, Shen HT. Domain adaptive remaining useful life prediction with transformer. *IEEE Trans Instrum Meas* 2022;71:1–13.
- [29] Miao M, Yu J, Zhao Z. A sparse domain adaption network for remaining useful life prediction of rolling bearings under different working conditions. *Reliab Eng Syst Saf* 2022;219:108259.
- [30] Sun M, Wang H, Liu P, Huang S, Wang P, Meng J. Stack autoencoder transfer learning algorithm for bearing fault diagnosis based on class separation and domain fusion. *IEEE Trans Ind Electron* 2022;69(3):3047–58.
- [31] Pan Y, Han Z, Wu T, Lei Y. Remaining useful life prediction of lubricating oil with small samples. *IEEE Trans Ind Electron* 2023;70:7373–81.
- [32] Chen J, Li D, Huang R, Chen Z, Li W. Aero-engine remaining useful life prediction method with self-adaptive multimodal data fusion and cluster-ensemble transfer regression. *Reliab Eng Syst Saf* 2023;234:109151.
- [33] Liu C, Qin C, Shi X, Wang Z, Zhang G, Han Y. TScatNet: An interpretable cross-domain intelligent diagnosis model with antinoise and few-shot learning capability. *IEEE Trans Instrum Meas* 2021;70:1–10.
- [34] Finn C, Abbeel P, Levine S. Model-agnostic meta-learning for fast adaptation of deep networks. 2017, ArXiv, arXiv:1703.03400.
- [35] Pu X, Li C. Meta-self-training based on teacher–student network for industrial label-noise fault diagnosis. *IEEE Trans Instrum Meas* 2023;72:1–11.
- [36] Hu Y, Liu R, Li X, Chen D, Hu Q. Task-sequencing meta learning for intelligent few-shot fault diagnosis with limited data. *IEEE Trans Ind Inf* 2022;18:3894–904.
- [37] Mo Y, Li L, Huang B, Li X. Few-shot RUL estimation based on model-agnostic meta-learning. *J Intell Manuf* 2023;34:2359–72.
- [38] Pan T, Chen J, Ye Z, Li A. A multi-head attention network with adaptive meta-transfer learning for RUL prediction of rocket engines. *Reliab Eng Syst Saf* 2022;225:108610.
- [39] Pan T, Zhang S, Li F, Chen J, Li A. A meta network pruning framework for remaining useful life prediction of rocket engine bearings with temporal distribution discrepancy. *Mech Syst Signal Process* 2023.
- [40] Feng Y, Chen J, Xie J, Zhang T, Lv H, Pan T. Meta-learning as a promising approach for few-shot cross-domain fault diagnosis: Algorithms, applications, and prospects. *Knowl-Based Syst* 2022;235:107646.
- [41] Song W, Duan S, Zio E, Kudreyko AA. Multifractional and long-range dependent characteristics for remaining useful life prediction of cracking gas compressor. *Reliab Eng Syst Saf* 2022;225:108630.
- [42] Rezende DJ, Mohamed S, Wierstra D. Stochastic backpropagation and approximate inference in deep generative models. In: *ICML*. 2014.
- [43] Ravi S, Larochelle H. Optimization as a model for few-shot learning. In: *ICLR*. 2017.
- [44] Shang J, Niu C, Huang J, Zhou Z, Yang J, Xu S, et al. Few-shot domain adaptation through compensation-guided progressive alignment and bias reduction. *Appl Intell* 2022;52(10):10917–33.
- [45] Fu S, Zhang Y, Lin L, Zhao M, Zhong S. Deep residual LSTM with domain-invariance for remaining useful life prediction across domains. *Reliab Eng Syst Saf* 2021;216:108012.
- [46] Chang Y, Li F, Chen J, Liu Y, Li Z. Efficient temporal flow transformer accompanied with multi-head probpase self-attention mechanism for remaining useful life prognostics. *Reliab Eng Syst Saf* 2022;226:108701.
- [47] Zhang Z, Chen X, Zio E, Li L. Multi-task learning boosted predictions of the remaining useful life of aero-engines under scenarios of working-condition shift. *Reliab Eng Syst Saf* 2023;237:109350.
- [48] Li W, Shang ZW, Gao M, Qian S, Feng Z. Remaining useful life prediction based on transfer multi-stage shrinkage attention temporal convolutional network under variable working conditions. *Reliab Eng Syst Saf* 2022;226:108722.